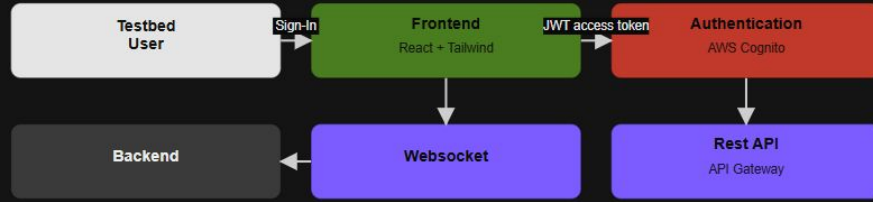


# **Healthcare/IoT Cybersecurity Testbed: Semester 2 Project Plan**

By: Justin Bower, Nathan Maloney, and Ipule Pipi

Instructors/Advisors: Dr. Sudhakaran and Dr. Aydeger

## FRONTEND

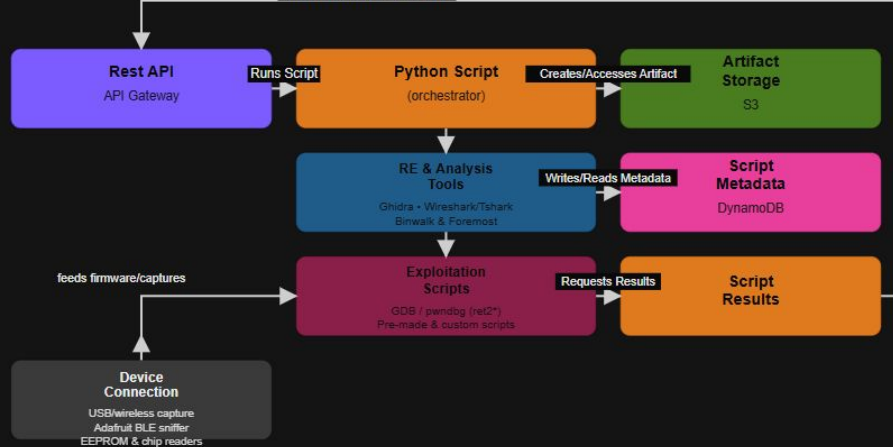


## STORAGE

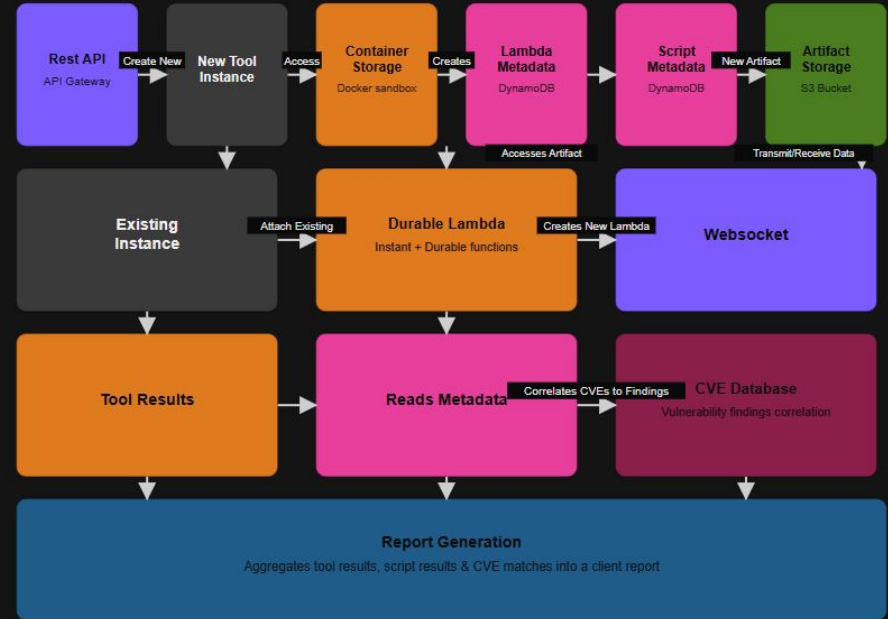


## SCRIPTS AND TOOLS

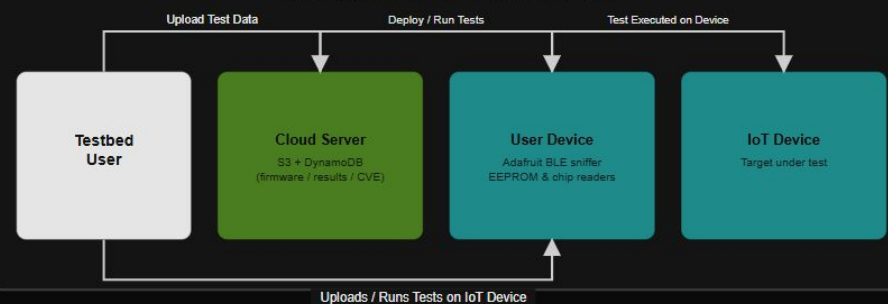
Requests Results → Rest API



## TOOLS



## SECURITY TESTING



## Testbed user

### Local testbed — client machine

#### Device connection

USB / wireless capture  
Adafruit BLE sniffer  
EEPROM & chip readers

#### RE & analysis tools

Ghidra  
Wireshark / Tshark  
Binwalk & Foremost

#### Exploitation scripts

GDB / pwndbg (ret2\*)  
Pre-made & custom scripts

### AWS cloud — server side

#### Web dashboard

React + Tailwind UI  
Cognito authentication

#### API + compute

API Gateway  
Lambda: instant + durable

#### Storage & sandbox

DynamoDB + S3 storage  
Docker sandboxed analysis  
CVE database

Uploads firmware, results & CVE data

# Frontend Integration- Device Management and Upload Workflow

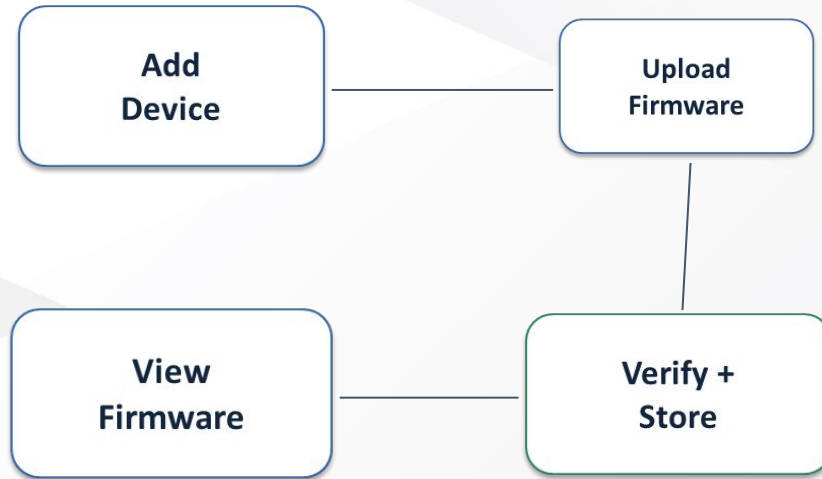
Authenticated dashboard workflow for creating and listing IoT/medical devices.

Firmware upload modal connected to the real backend and S3 upload path.

Backend confirmation step makes upload completion authoritative.

Firmware viewing/refetch workflow exposes the stored artifact state.

Retry/concurrency design protects against stale upload attempts.



# Device Creation Flow

A device is created once, then becomes the parent for firmware artifacts.



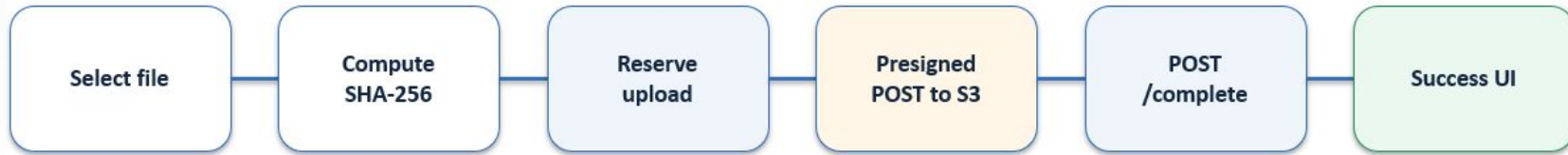
Device metadata uses `DEVICE#<deviceId> / METADATA`.

Relationship rows associate the authenticated user with the device.

That relationship lets later list operations return only the user's devices.

# Firmware Upload: End-to-End

The key design is a two-stage upload: reserve metadata first, then upload the binary directly to S3.



25 MiB client-side size cap; Web Crypto computes SHA-256.

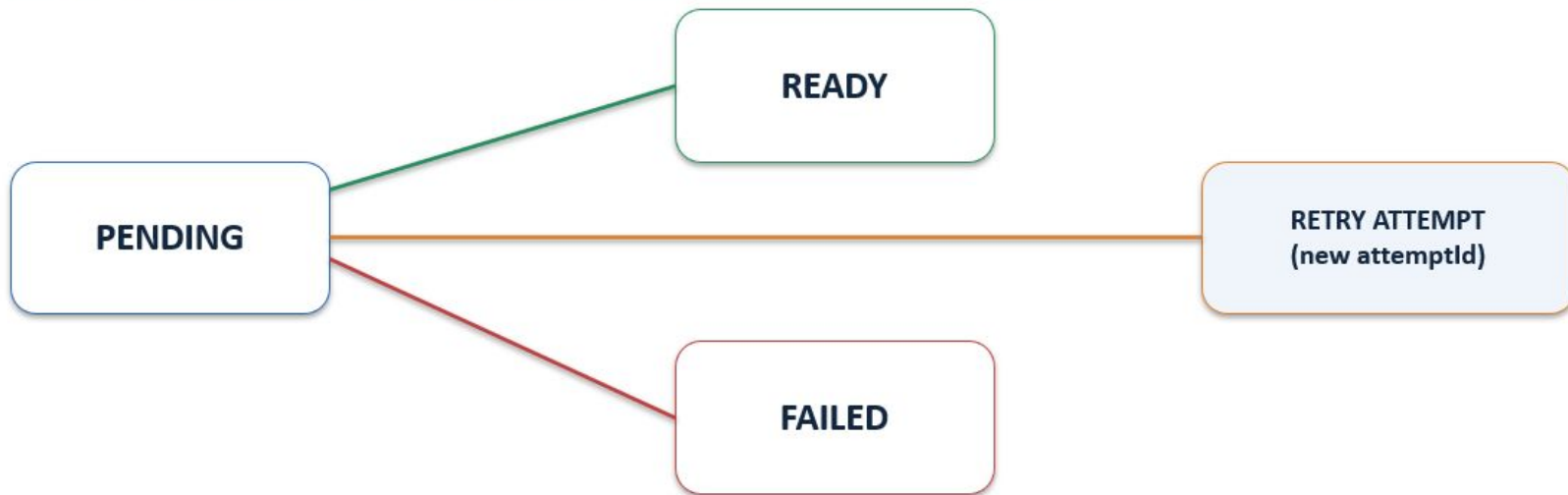
Reservation records immutable firmware metadata and creates an upload attempt.

The binary bypasses Lambda, reducing backend transfer/load.

/complete checks the uploaded S3 object against the reserved metadata.

# Firmware State & Retry Semantics

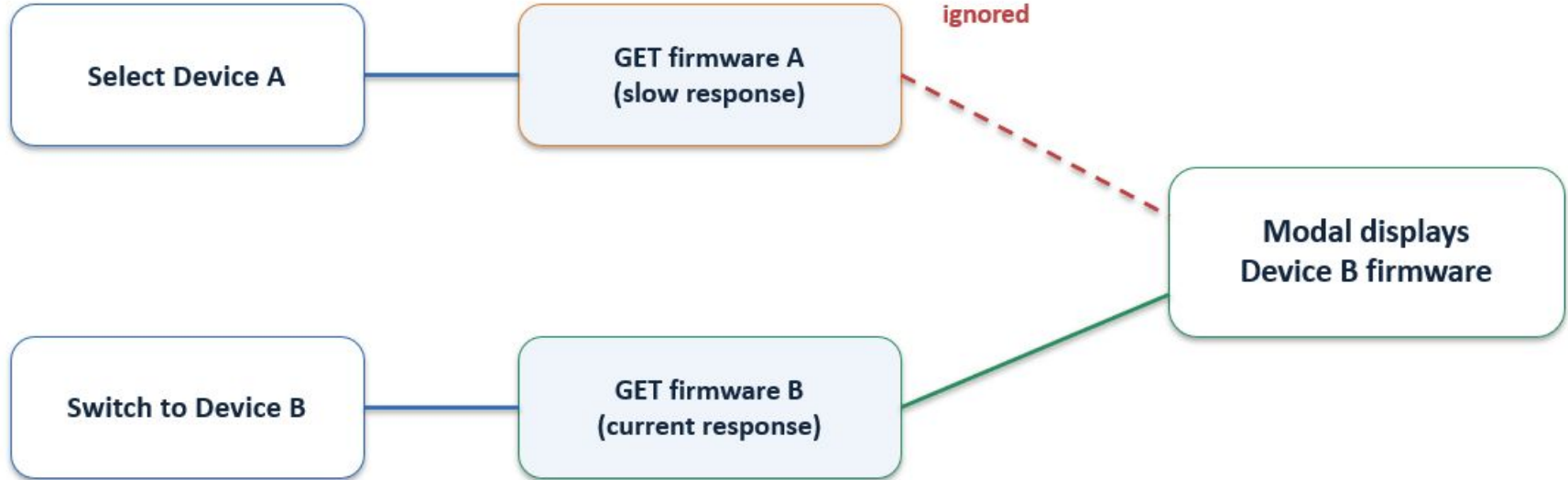
Retry is designed for an unfinished pending attempt—not to repair immutable metadata that was proven wrong.



Important: retry preserves firmwareId, filename, version, sizeBytes and sha256. retryOfAttemptId identifies which attempt the client is retrying, protecting against stale clients/concurrent attempts.

# View Firmware: Avoiding Stale UI Results

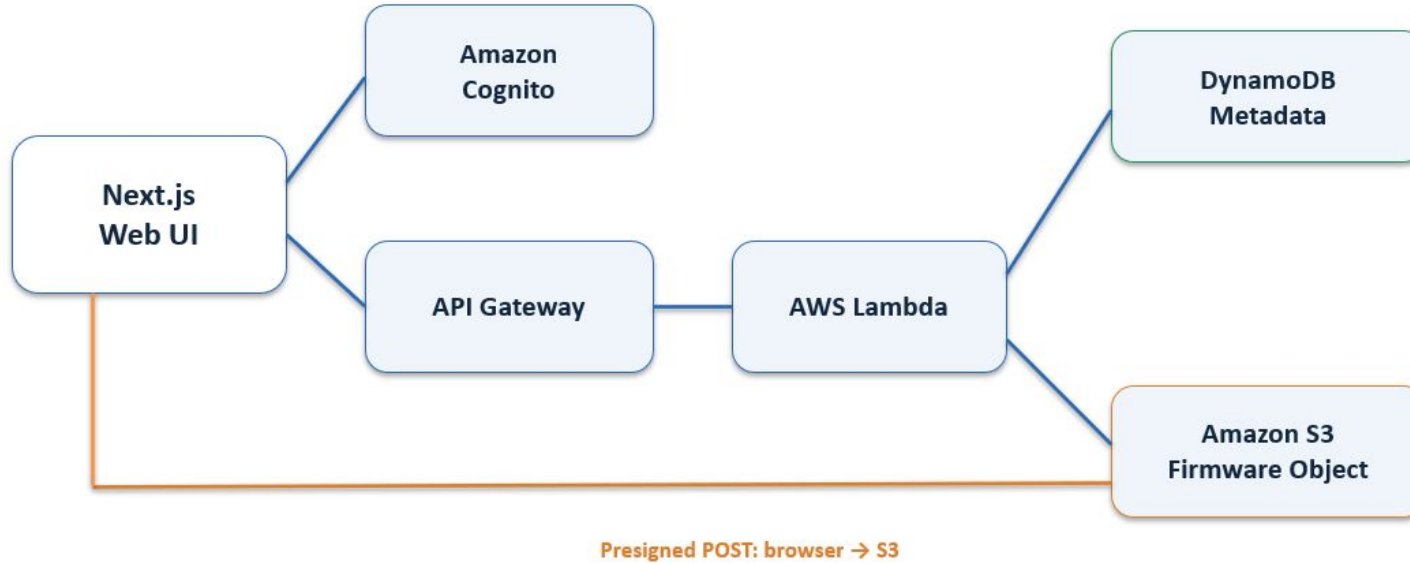
The modal refetches for the selected device, and cleanup prevents an older request from overwriting a newer selection.





# Frontend and AWS System Architecture

The browser does not directly own infrastructure logic; it coordinates authenticated APIs and a presigned S3 upload.



# Web Interface Demo - Device Management & Firmware Upload



# Frontend Integration - Key Features

The implementation is more than a file-upload button: the workflow preserves security, consistency, and testbed isolation.

Security: authenticated API calls; browser receives temporary presigned upload permission rather than AWS credentials.

Scalability: firmware binaries go directly to S3; Lambda manages metadata and verification.

Integrity: SHA-256 + size are reserved and later verified before the firmware becomes ready.

Concurrency: attemptId / retryOfAttemptId prevents a stale client from blindly retrying the wrong upload attempt.

Consistency: /complete is authoritative for the upload action; list/view is a fresh read for display.

Testbed relevance: each stored firmware artifact can become the reproducible input for later isolated vulnerability-analysis environments.

# Key System Features

- Users can connect a variety of IoT and medical devices that use standard interfaces, wired or wireless, to the testbed, enabling full device access, interaction, and manipulation.
- Users can extract firmware directly from connected IoT and medical devices, obtaining complete firmware/binaries for detailed examination and vulnerability assessment.
- Users can analyze firmware entropy to evaluate randomness and compression characteristics
- Users can utilize industry-standard open-source tools to perform reverse-engineering tasks and comprehensive vulnerability research.
- Users can input their own CVE lists, and automatically retrieve lists of known CVEs, which are stored server-side in a database.
- Users can utilize pre-made scripts or upload custom exploitation scripts to rapidly test known, suspected, or new vulnerabilities, and potential attack vectors.
- Users can perform all connection, extraction, analysis, and exploitation tasks through an intuitive client-side graphical interface, thereby streamlining the process for both novice and experienced users.
- Users can create and upload documentation, or integrate existing documentation via GitHub, with all materials stored in server-side storage.

# What makes this novel?

- A testbed that tightly integrates many disparate tools, automates firmware extraction, firmware analysis, entropy analysis, decryption, into one platform, is a novel approach to cybersecurity analysis.
- The testbed reduces the background knowledge needed for manufacturers or researchers to reverse-engineer Medical/IoT devices while unifying the reverse-engineering toolset.
- The testbed is specifically tailored for IoT and medical hardware, which is often proprietary and notoriously difficult to analyze.
- The testbed allows for direct firmware extraction from connected devices via standard interfaces and therefore streamlines the process of obtaining firmware/binaries for vulnerability assessment.
- The testbed allows for built-in entropy analysis combined with pre-existing decryption techniques to uncover encrypted or obfuscated firmware sections more efficiently.

# What makes this novel?

- The testbed has support for both ready-made exploitation scripts and user-developed custom scripts provides flexibility to confirm known weaknesses or explore new attack vectors.
- The testbed will have backend/web support for supporting the local GUI that lowers the barrier to entry so that both novice and experienced users can perform complex tasks with greater ease.
- The testbed will have documentation storage and a CVE database with associated automations, accelerating the research process via targeted testing, and expedited sharing of results
- The testbed's benefits scale with the difficulty of reverse-engineering each device and contribute to a more secure future for IoT and Medical devices by making thorough examination more accessible.

# What tools we will be using?



[USB EEPROM Programmer](#)

## *Reverse Engineering Tools:*

- Ghidra: Open source tool for binary/firmware analysis
- Wireshark/Tshark (GUI and CLI): Open source tool for network analysis
- Binwalk/Foremost CLI: Useful for examining files, extracting sub-files, and entropy analysis
- GDB/pwndbg: Useful for low-level Assembly-language reverse engineering, debugging, and vulnerability research
- Python: Ideal for creating scripts that interface with aforementioned tools via libraries and APIs
- Adafruit nRF51822-based Bluefruit LE Sniffer: USB device, drivers, and matching API for network capture (Bluetooth traffic logging as pcap files)
- Various Wireless Adapters: USB device, drivers, and matching API for network capture (Wifi/RF traffic logging as pcap files)
- EEPROM and Live Chipset Reader: Useful for pulling firmware directly off PCBS
- Soldering irons: For connecting and disconnecting electrical connections
- Solder wiring (lead or non lead): For establishing and holding electrical connections
- Copper wick: For wiping unused/excess solder
- Alligator clips: For holding PCBs and providing temporary connections where soldering is unnecessary

# What tools we will be using?



## *Frontend and Backend Tools:*

- React: Frontend of website
- TailwindCSS: Styling of website
- Terraform: Deployment of services to cloud in a reproducible fashion that allows for easy transfer of the stack to a different AWS account
- Docker: Isolates tasks for security, ease of deployment and platform management

Amazon Web Services (AWS): Provides web server hosting, database backend, and other miscellaneous compute needs listed below:

- DynamoDB: Database architecture backbone; keeps data organized, minimizes storage space complexity and computational overhead via data structuring
- S3 Object Storage: Simple artifact storage and web hosting
- Instance Lambda Functions: Used to optimize speed of repetitive tasks and minimize memory space complexity
- Durable Lambda Functions: Allows for easier control of long running cloud analysis that may involve waiting for user input
- API Gateway: Allows for a secure connection between local and cloud applications
- Cloudfront: Improved web latency and increased max traffic capacity
- Cognito: AWS managed sign on portal with authentication tokens and account management



# Milestone 4 Task Matrix

| Team Member | Main Responsibility Category                                                                     | Sub Task 1                                                                             | Sub Task 2                                                                             | Sub Task 3                                                                                                                           | Sub Task 4                                                                                                                                                 | Sub Task 5                                                       |
|-------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Justin      | Reverse Engineering, Vulnerability Research, Cyber Tooling/ Demos, Local GUI, Project management | Prepare a demo of the Adafruit device and pcap file extraction using Wireshark/ Tshark | Complete CVE Research for wifi chipsets and enter said CVEs into the database for demo | Study I2C sensor and plan integration into project, such as an oximeter demo, and planning how it will interface in the main project | Respond to professors emails, communicate/ coordinate with team members, attend professor meetings, create/write most of the Project Plan and Presentation | Wipe the lab computer and install Linux (Got permission already) |
| Nathan      | Web/Database Backend and Demos                                                                   | Create the infrastructure to allow for cloud based analysis                            | Prepare a demo of cloud based analysis of device attack results                        | Integrate artifact storage and database with local and cloud infrastructure                                                          | Create secure API endpoints to allow local and cloud applications to interact                                                                              | Setup router for testbed and interfacing devices in the network  |

|        |                                             |                                                                                                                             |                                                                                                                                                           |                                                                                                                                                                                                             |                                                                                                                                                                                                         |                                                                                                                                                                                                              |
|--------|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ipule  | Web Front End and Cloud Backend Integration | Complete the authenticated device management workflow for adding and viewing IoT/medical devices through the web dashboard. | Integrate device creation and retrieval with AWS API Gateway, Lambda, DynamoDB, and Cognito so device records are associated with the authenticated user. | Implement the end-to-end firmware upload workflow, including firmware reservation, SHA-256 metadata, presigned S3 upload, upload completion verification, and retry support for interrupted upload attempts | Implement the firmware viewing interface so users can retrieve and display firmware metadata, upload status, filename, version, file size, and other associated information for each registered device. | Strengthen firmware upload state management and authorization by using firmware/attempt identifiers to prevent stale or conflicting retries and ensure users can access only their own devices and firmware. |
| Jordan | RE/VR, Cyber Tooling/ Demos, Local GUI      | Continue local GUI work so users can trigger the RE/analysis/exploitation tools                                             | Continue developing pre-made exploitation scripts for GDB/pwndbg (ret2*)                                                                                  | Develop pre-made exploitation scripts for Ghidra and/or integrate existing Ghidra scripts                                                                                                                   | Begin live firmware extraction using EEPROM/chipset readers                                                                                                                                             | Install necessary tools on Linux machine                                                                                                                                                                     |



# Milestone 5 Tasks

## Ipule - Web Front End and Cloud Backend Integration

- Integrate the firmware-analysis workflow into the web dashboard so a user can select an uploaded firmware image and initiate an analysis job.
- Develop frontend interfaces for viewing analysis jobs, execution status, and generated results associated with a specific device and firmware version.
- Integrate the dashboard with the backend analysis APIs and storage architecture developed by the team, including retrieval of analysis outputs and artifacts.
- Improve device and firmware workflow state handling by displaying clear pending, ready, processing, completed, and failed states where applicable.
- Implement user interface protections and validation for analysis actions so jobs can only be started against valid firmware owned by the authenticated user.

## Nathan - Web/Database Backend and Demos

- Create proof of concept docker/sidecar deployments are properly set up with the tooling of the DS18B20 I2C sensor work
- Starting working for DS18B20 I2C sensor (and similar sensor's) exploitation can be verified by a new user through their own replication of the pipeline
- Testing security of tooling and platform itself, as part of the containerization and isolation of environment

# Milestone 5 Tasks

Justin - RE/VR, Cyber Tooling/Demos, Local GUI, Project Management

- Begin integrating Adafruit and Wireshark/Tshark
- Continue development of pre-made exploitation scripts for wifi/bluetooth exploitation
- Keep project/team members moving and ensure Professors (customers) are happy
- Maintain email/Discord communications constantly
- Create intermediate start-to-finish proof of concept demo

Jordan - RE/VR, Cyber Tooling/Demos, Local GUI

- Continue local GUI work so users can trigger the RE/analysis/exploitation tools
- Build out Ghidra-based reverse engineering workflows for extracted firmware, producing analysis outputs
- Begin integrate entropy analysis and file segmentation tools (Binwalk/Foremost)
- Continue development of pre-made exploitation scripts for wifi/bluetooth exploitation
- Begin I2C sensor integration into project

# Milestone 6 Tasks

## Ipule - Web Front End and Cloud Backend Integration

- Complete the end-to-end web workflow from device registration and firmware upload through analysis execution and result retrieval.
- Integrate available reverse-engineering and vulnerability-analysis outputs into the dashboard, such as extracted firmware information, entropy results, discovered files, vulnerabilities, or generated artifacts provided by the analysis backend.
- Improve the dashboard user experience by organizing devices, firmware versions, analysis jobs, and outputs into a clear workflow suitable for researchers and manufacturers.
- Perform integration and edge-case testing of authentication, device ownership, firmware upload/retry behavior, analysis initiation, and result retrieval.
- Prepare the web application for the final project demonstration by resolving integration issues, improving error handling, and documenting/demoing the complete user workflow.

## Jordan - RE/VR, Cyber Tooling/Demos, Local GUI

- Finalize device connections, to firmware extractions, to entropy/decryption, to exploitation/analysis paths, creating multiple local pipelines that feeds results to the web backend
- Finalize local GUI work so users can trigger the RE/analysis/exploitation tools locally
- Complete and test the exploitation scripts against real device/firmware data
- Prepare demos of the cyber tooling integrations for the final presentation
  - GDB/pwndbg
  - Ghidra demo

# Milestone 6 Tasks

Justin - RE/VR, Cyber Tooling/Demos, Local GUI, Project Management

- Finalize CVE entries and RE documentation for the final demo
- Prepare demos of the cyber tooling integrations for the final presentation
  - Adafruit/Wireshark
  - Binwalk/Foremost
- Prepare a full end-to-end RE demo (through aforementioned local pipelines) for the final presentation as a video recording
- Keep project/team members moving to ensure Professors are satisfied with progress
- Maintain email/Discord communications constantly with team members and Professors

Nathan

- Finalize AWS permissions and backend security using least-privilege access.
- Complete integration between local analysis tools, cloud services, S3, DynamoDB, and the web application.
- Finalize and test containerized cloud analysis and isolation.
- Perform end-to-end testing and prepare the cloud/backend workflow for the final demo.

**Thanks for listening!**  
**Questions?**